Attention Mechanism

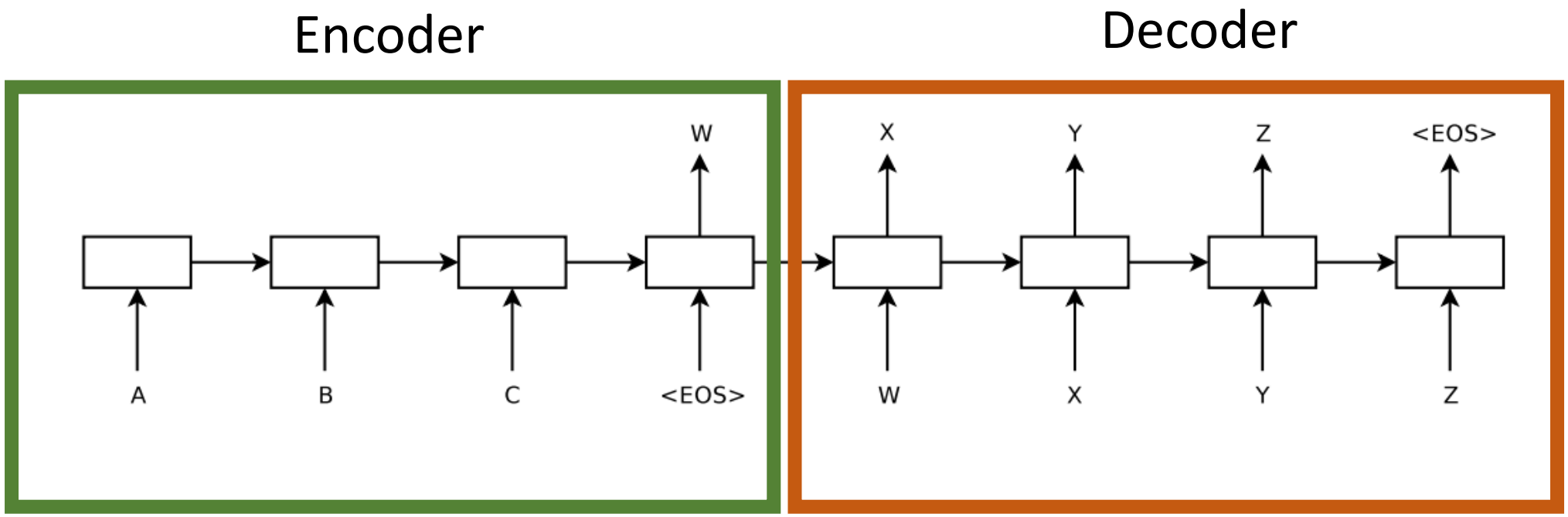
Natural Language Processing

Ayesha Enayet

Background

- ANNs can only be applied to problems whose inputs and targets can be sensibly encoded with vectors of fixed dimensionality.
  - many important problems are best expressed with sequences whose lengths are not known a-priori.
  - question answering can also be seen as mapping a sequence of words representing the question to a sequence of words representing the answer.
- Sequences pose a challenge for DNNs because they require that the dimensionality of the inputs and outputs is known and fixed.

Encoder-Decoder Models



Sequence to Sequence Learning with Neural Networks: https://arxiv.org/pdf/1409.3215

RNN Encoder–Decoder (Cho et al., 2014)

- One RNN encodes a sequence of symbols into a fixed length vector representation, and the other decodes the representation into another sequence of symbols (variable length).
- learn the conditional distribution over a variable-length sequence conditioned on yet another variable-length sequence, e.g. $p(y_1, \ldots, y_T \mid x_1, \ldots, x_T)$,

Drawbacks

- Drawback: Only final hidden state (context vector) passed on to the decoder.
- Solution: Attention.

Attention (RNN based)

- First, the encoder passes *all* the hidden states to the decoder.
- Second, an attention decoder does an extra step before producing its output. In order to focus on the parts of the input that are relevant to this decoding time step, the decoder does the following:
  1. Look at the set of encoder hidden states it received – each encoder hidden state is most associated with a certain word in the input sentence
  2. Give each hidden state a score (let's ignore how the scoring is done for now)
  3. Multiply each hidden state by its softmaxed score, thus amplifying hidden states with high scores, and drowning out hidden states with low scores

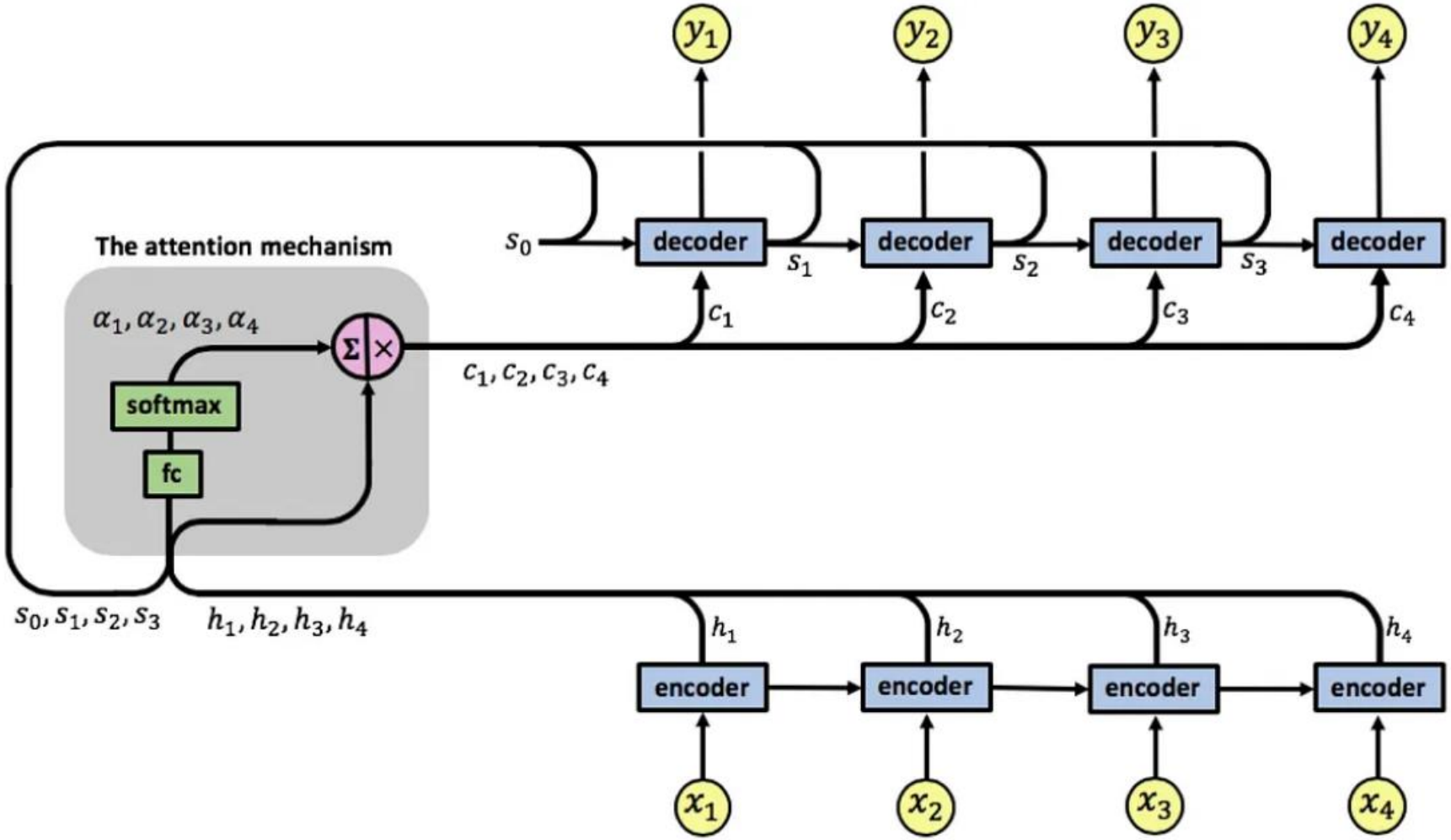
Attention Scores

- Cab be computed:
  1. The attention score can be computed through cosine similarity, i.e., the dot product of two-word vectors, which assesses the strength of their relationship or the degree of similarity between the compared words.
  2. Feedforward Neural Network.

Context Vector: $C(i) = \sum_{j=1}^{T_x} \alpha\ (i,\ j)\ h(j)$

Score: $e_{i,j} = f(s_{i-1}, h_j)$

Attention Score (SoftMax of score): $\alpha_{i,j} = \frac{exp(e_{i,j})}{\sum_{k=1}^{T} exp(e_{i,k})}$


y_1
y_2
y_3
y_4
The attention mechanism
s_0
decoder
decoder
decoder
decoder
s_1
s_2
s_3
$\alpha_1, \alpha_2, \alpha_3, \alpha_4$
Σ
$\times$
c_1
c_2
c_3
c_4
c_1, c_2, c_3, c_4
softmax
fc
s_0, s_1, s_2, s_3
h_1, h_2, h_3, h_4
h_1
h_2
h_3
h_4
encoder
encoder
encoder
encoder
x_1
x_2
x_3
x_4

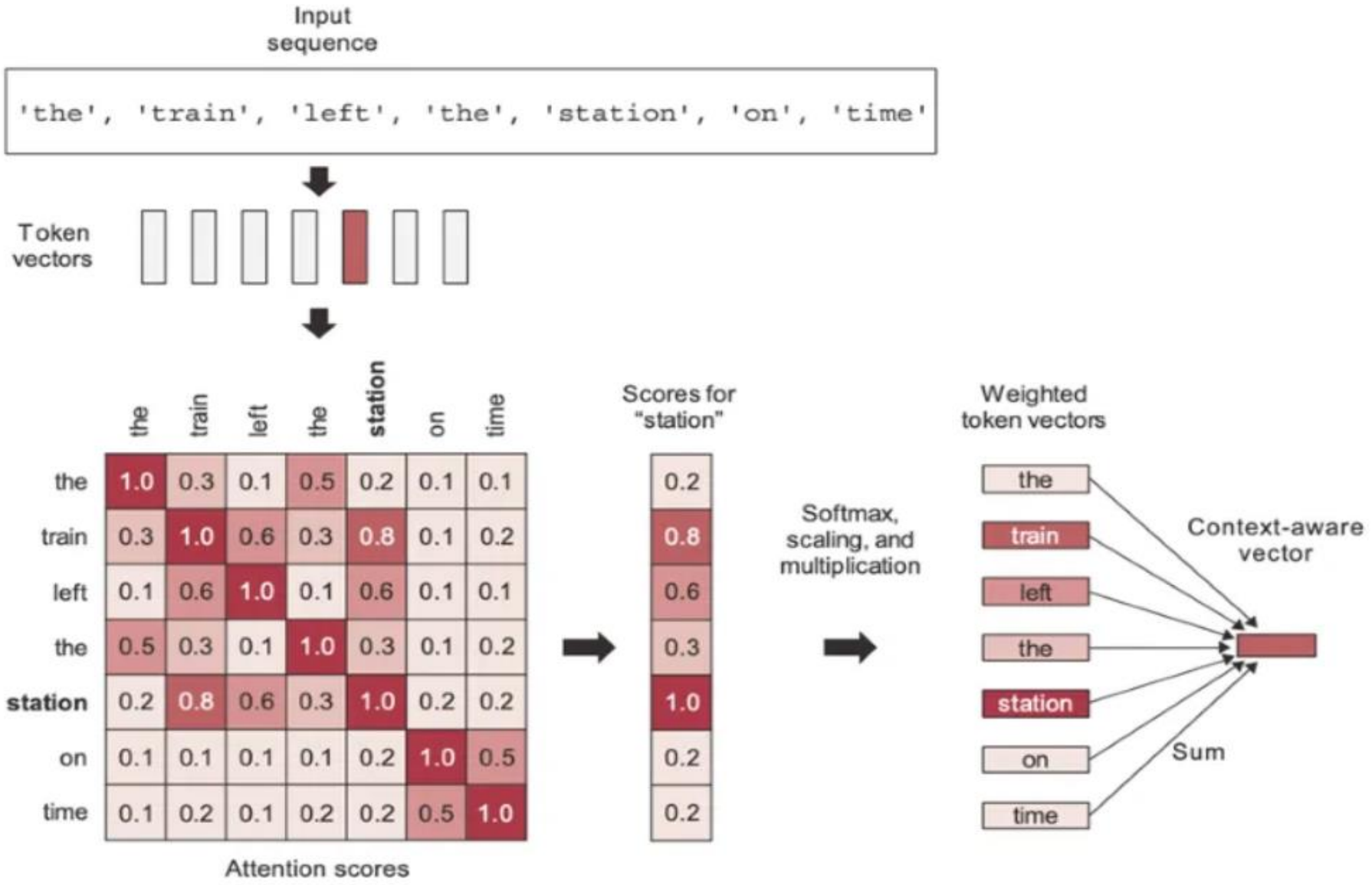
Types of Attention

- Self-Attention:
  - Focuses on relationships within the same sequence.
  - Encoder, Decoder
- Cross-Attention:
  - Enables the model to consider relationships between different sequences (e.g., input and output).
  - Decoder

Reference: https://medium.com/@abhinavbhartigoml/unraveling-transformers-a-deep-dive-into-self-attention-and-3e37dc875bea#:~:text=Self%2DAttention%3A%20Focuses%20on%20relationships,e.g.%2C%20input%20and%20output).

Self Attention

- ht
  d8


Input sequence
'the', 'train', 'left', 'the', 'station', 'on', 'time'
Token vectors
the train left the station on time
the 1.0 0.3 0.1 0.5 0.2 0.1 0.1
train 0.3 1.0 0.6 0.3 0.8 0.1 0.2
left 0.1 0.6 1.0 0.1 0.6 0.1 0.1
the 0.5 0.3 0.1 1.0 0.3 0.1 0.2
station 0.2 0.8 0.6 0.3 1.0 0.2 0.2
on 0.1 0.1 0.1 0.1 0.2 1.0 0.5
time 0.1 0.2 0.1 0.2 0.2 0.5 1.0
Attention scores
Scores for "station"
0.2
0.8
0.6
0.3
1.0
0.2
0.2
Softmax, scaling, and multiplication
Weighted token vectors
the
train
left
the
station
on
time
Context-aware vector
Sum

Self Attention

- Self-attention, sometimes called intra-attention is an attention mechanism relating different positions of a single sequence in order to compute a representation of the sequence. Self-attention has been used successfully in a variety of tasks including reading comprehension, abstractive summarization, textual entailment and learning task-independent sentence representations.

Table 1: Comparison of Self attention and RNN based Attention

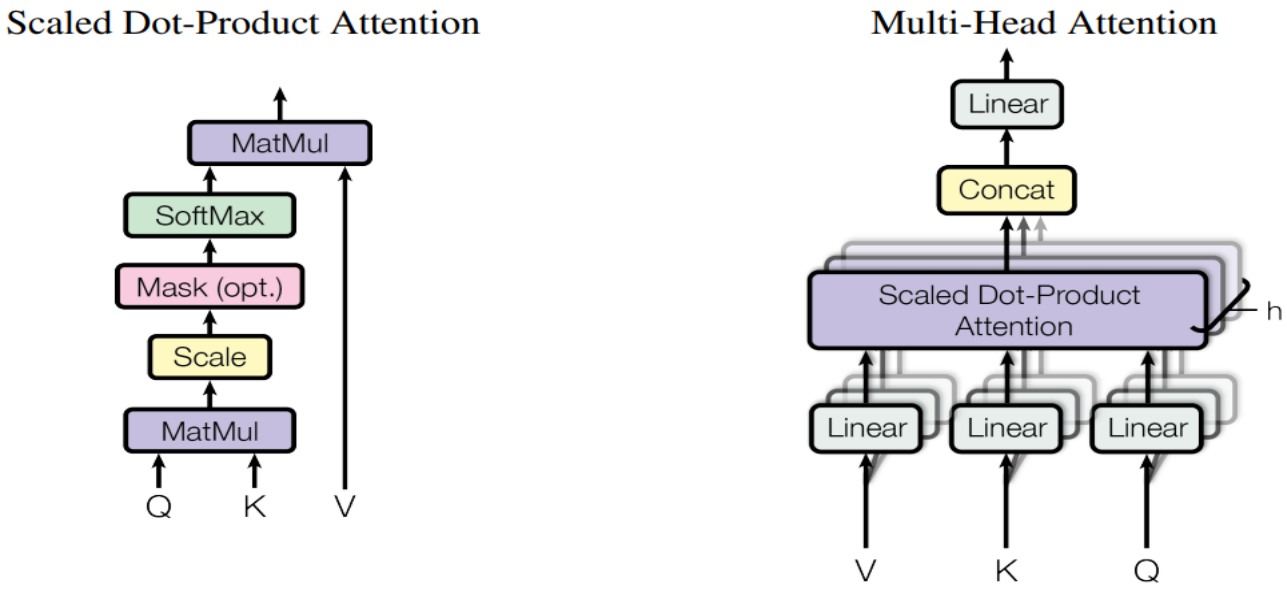
RNN Based Attention Mechanism	Self-Attention Mechanism
score: $e_{i,j} = f(s_{i-1}, h_j)$	**score** $= x_i^T x_j$ [Dot product of input vector with itself.]
Attention Score (SoftMax of score**):** $\alpha_{i,j} = \frac{exp(e_{i,j})}{\sum_{k=1}^{T} exp(e_{i,k})}$	**Attention score:** $\alpha_{i,j}$ = SoftMax(score/const.) = SoftMax $(\frac{x_i^T x_j}{const})$ [Dividing by a constant to control the variance.]
Context Vector: C(i)= $\sum_{j=1}^{T_x} \alpha\ (i,\ j)\ h(j)$	**Self-Attention Vector**: A(i)= $\sum_{j=1}^{T_x} \alpha\ (i,\ j) x_j$ = SoftMax $(\frac{x_i^T x_j}{const})$. x_j [in vectorized form]

Drawback

- The drawback of previously proposed self-attention:
  - We computed the Self-Attention vector, as derived above, based on the inputs of the vectors themselves.
  - In other words, there are no learnable parameters.

Solution

- “Attention is all you need”: **three trainable weight matrices are introduced and multiplied with input Xi separately, and three new terms Queries(Q), Keys(K), and Values(V)** come into the picture.



Solution

$shape\ of\ X \rightarrow (T \times d_{model})$

d_{model} = Embedding vector for each word (512 as per the paper)

$Queries$: $\boldsymbol{Q} = \boldsymbol{X}W^{Q}$; *Where* $\boldsymbol{W^{Q}}$ *is weight matrix introduced to calculate Q from X.*

$Keys$: $\boldsymbol{K} = \boldsymbol{X}W^{K}$; *Where* $\boldsymbol{W^{K}}$ *is weight matrix introduced to calculate K from X.*

Values: $\boldsymbol{V} = \boldsymbol{X}W^{V}$; Where $\boldsymbol{W^{V}}$ is weight matrix introduced to calculate V from X.

$$\textbf{\textit{Shape tracking}} \rightarrow (\textbf{\textit{shape of X}}) \ \textbf{\textit{Matrix Mult.}} (\textbf{\textit{shape of W}}^{-}) \rightarrow \textbf{Output shape}$$

$$Q = XW^{Q} \rightarrow (T \times d_{model}) * (d_{model} \times d_k) \rightarrow (T \times d_k)$$

$$K = XW^{K} \rightarrow (T \times d_{model}) * (d_{model} \times d_k) \rightarrow (T \times d_k)$$

$$V = XW^{V} \rightarrow (T \times d_{model}) * (d_{model} \times d_v) \rightarrow (T \times d_v)$$

Finally, substituting Q, K, V, and const. = $\sqrt{d_k}$ (d_k dimensionality of the key vector) in above self-attention eqn. (I), we get equation for attention as given below:

$$\textbf{Attention (Q, K, V) = SoftMax } \left(\frac{\boldsymbol{QK^T}}{\sqrt{\boldsymbol{d_k}}}\right)\textbf{V} \quad \text{--------- (II)}$$

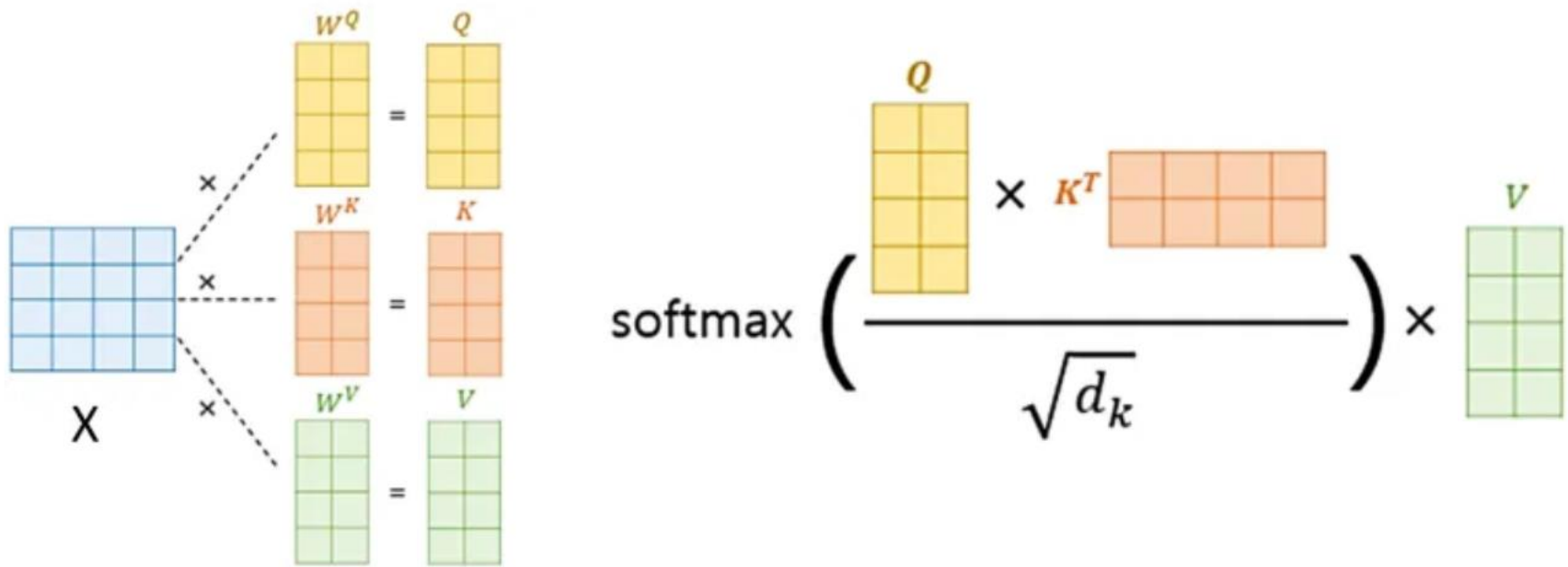


Figure 5. (Ref.)

Shapes as per the paper

Object	*Shape*	*values*
q_i, k_i	d_k	$(64,)$
v_i	d_v	$(64,)$
x_i	d_{model}	$(512,)$
W^Q, W^K	$d_{model} \times d_k$	$(512, 64)$
W^V	$d_{model} \times d_v$	$(512, 64)$

Shape of $QK^T \rightarrow (T \times d_k) \times (d_k \times T) \rightarrow (T \times T)$

And finally, the shape of final attention output: $(T \times T) \times (\mathrm{T} \times d_v) \rightarrow (\mathrm{T} \times d_v)$

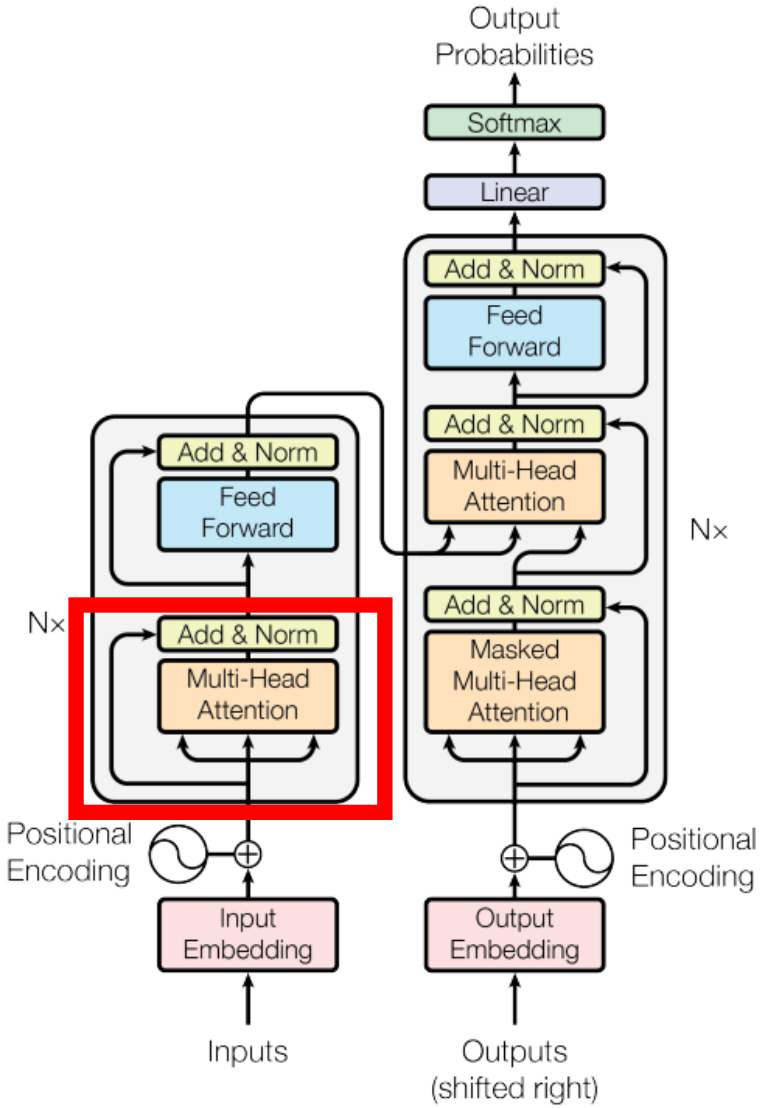


Figure 1: The Transformer - model architecture.

- https://www.youtube.com/watch?v=QCJQG4DuHT0
- https://medium.com/@ramendrakumar/self-attention-d8196b9e9143
- https://medium.com/@geetkal67/attention-networks-a-simple-way-to-understand-self-attention-f5fb363c736d